Quiz 7: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 7 is Thursday Oct 15, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Scope

The Oct 8 lecture, and nothing past where it stopped. That meeting covered the back half of the CNN deck and then the detection deck as far as non-maximum suppression. The quiz stops in the same place.

If you are revising from the slides, that means: the Week 6 deck from "Building a CNN" to the end, and the Week 7 deck from the start through "Non-maximum suppression."

On the quiz:

- The CNN skeleton: backbone, downsampling, head, and what each part is for
- Downsampling by pooling against downsampling by stride
- Getting from a feature map to a vector: flatten, global average pooling, or a 1×1 convolution
- Channel arithmetic: why a filter's depth is forced, and what sets the number of output channels
- Feature maps, and why they are not colour channels
- 1×1 convolutions and what they are used for
- Batch normalization, including how `train()` and `eval()` differ
- Residual connections, and why they let networks get deep
- Dense connections, and how concatenating differs from adding
- Grouped and depthwise separable convolutions, and the saving they buy
- The architecture tour, at the level of what each one contributed
- Transfer learning: feature extraction against fine-tuning
- The input contract, and why each pretrained model has its own preprocessing
- When transfer learning stops helping
- Backbone, neck, head
- The four tasks, and what a label looks like for each
- Why detection is harder than classification
- Intersection over Union, including the arithmetic
- The region proposal idea, R-CNN, selective search
- Non-maximum suppression, and why it is done per class

On the quiz from Lab 6, which was due the day before:

- How many parameters a frozen ResNet-18 with a new 10-class head trains
- What the detector's own `transforms()` reported, and why you use it
- What the score threshold does, and what NMS does
- Why one duplicate box survived per-class NMS

Not on the quiz. Anything the lecture did not reach: Fast R-CNN, Faster R-CNN, ROI pooling and ROI Align, anchor boxes and box offsets, YOLO and one-stage detection, feature pyramid networks, mAP, how segmentation models are built (transposed convolution, U-Net, segmentation losses, Mask R-CNN), and Vision Transformers and DETR. All of that moves to Quiz 8. You are still responsible for knowing *what* instance segmentation is, because the four tasks were covered; you are not responsible for how an instance segmentation model works.

Part 1: The CNN skeleton

Almost every convolutional classifier has the same three parts.

The backbone (also called the body) is repeated convolution layers with downsampling in between. It takes an image and turns it into a stack of feature maps. All the feature extraction happens here.

Downsampling sits between blocks of the backbone and reduces the width and height. You want this to happen: it is what lets the receptive field grow and the channel count rise without the tensor becoming enormous.

The head is a small network, usually fully connected, that does the actual task. Classification, regression, however many classes you like.

The reason this matters more than any individual architecture is that every computer vision task in the course is this skeleton with a different head on it. Change the head, change the task, leave the backbone alone.

Through the backbone, width and height shrink and the channel count grows. The amount of information carried stays roughly equivalent; what changes is what kind of information it is. Early layers have small receptive fields and describe edges and textures, with good spatial precision and little idea what the object is. Deep layers have large receptive fields and describe objects, with the spatial precision mostly gone.

Pooling against stride

Both halve the width and height. An 8×8 input becomes 4×4 either way.

The difference is that a strided convolution learns how to downsample and pooling does not. `MaxPool2d(2)` has zero parameters; it applies a fixed rule, take the largest value in each window. `Conv2d(64, 64, 3, stride=2)` has 36,928 parameters, so the network learns which combination of inputs to carry forward.

Max pooling is the more common of the two pooling variants. Average pooling exists and is used mostly at the very end of the network, for a different reason, described next.

Getting to a vector

A fully connected layer needs a vector, and the backbone produces something like $512 \times 7 \times 7$. Three ways across that gap:

Flatten. Read the whole tensor out as one long vector. For $7 \times 7 \times 512$ that is 25,088 numbers, and feeding a 4,096-unit layer then costs $25{,}088 \times 4{,}096 + 4{,}096 = 102{,}764{,}544$ parameters. VGG-16 does this, and it is why 102 of VGG's 138 million parameters sit in one layer that is not doing the interesting work.

Global average pooling. Collapse each feature map to a single number by averaging it. $7 \times 7 \times 512$ becomes 512 numbers, and the same 4,096-unit layer now costs $512 \times 4{,}096 + 4{,}096 = 2{,}101{,}248$ parameters, about 49 times less. Every architecture after VGG does this. The intuition for why it works: by the last layer a channel is already encoding something like "is there a cat here," so averaging it over space is a reasonable summary rather than a loss.

A 1×1 convolution. Also possible, and discussed below.

Part 2: Channels, filters and feature maps

The channel arithmetic

Two rules, and almost every shape question comes down to them.

A filter's depth is forced by the input. If the input has C channels and you choose a 3×3 kernel, each filter is $3 \times 3 \times C$. You do not get to choose the depth. For an RGB image that is $3 \times 3 \times 3$.

The number of filters is what you choose, and it sets the number of output channels. 64 filters produce 64 output channels. Typical counts in practice are 32, 64, 128 and so on upward as the spatial size comes down.

So a layer taking 64 channels with 32 filters of 3×3 has 32 filters, each of shape $3 \times 3 \times 64$, and produces 32 output channels.

Feature maps are not colour channels

The input to the network really is red, green and blue. The moment the first filter runs, that stops being true. Each output channel is now whatever that filter learned to respond to, and we call it a **feature map**.

This is worth getting straight because the vocabulary hides it. A feature map looks like an array of numbers that mostly will not mean anything to you. Early in the network a feature map might be an edge detector's response, something a Sobel filter would produce. Late in the network it might be closer to "is this a cat." You would not find an edge detector in a late layer; it would have nothing useful to do there.

1×1 convolutions

A convolution whose kernel is a single pixel. Each filter is $1 \times 1 \times C$, so it has one weight per input channel and looks at one spatial position at a time.

Because it cannot see any neighbours, it does not detect spatial patterns at all. What it does is **mix information across channels**. At each position it takes a learned weighted combination of all C channel values and produces one number.

The main use is changing the channel count cheaply. To go from 64 channels to 32 you need **32 filters, each** $1 \times 1 \times 64$, which is 2,048 weights total. Each filter learns which combination of the 64 inputs is worth keeping, which is better than discarding half the channels. Width and height are untouched.

This operation comes back repeatedly: in Inception, in ResNet bottlenecks, and as the "pointwise" half of a depthwise separable convolution.

Part 3: Batch normalization

In spreadsheet data you normalize across a feature, like a dog's weight. In image data you normalize **across a channel**: take all the values in the red channel, find their mean and variance, and standardize.

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

The ϵ is there to keep the denominator away from zero.

It is done per mini-batch during training. A batch is the group of examples you push through together; 32 images at a time means a batch size of 32. The mean and variance come from that batch.

`train()` against `eval()`

This distinction is examinable and it is easy to get backwards.

In `train()` mode, BatchNorm uses the statistics of the batch in front of it. A consequence worth sitting with: one image's output depends on which other images happen to share its batch. Change its batch-mates and its output changes.

In `eval()` mode, BatchNorm uses a running estimate of the mean and variance accumulated over training, and ignores the batch entirely. The same image gets the same answer regardless of what it is batched with.

That is why `eval()` is not optional at inference. Note also that `torch.no_grad()` does **not** do this for you. It controls whether gradients are recorded; it has nothing to do with layer mode.

If you are using a pretrained backbone, use the normalization constants it was trained with, not your own. More on that in Part 6.

Part 4: The connections that made depth work

The degradation problem

By 2015 the obvious move was more depth, and it stopped working. Very deep networks were painful to train because the gradient had to survive the trip back to the early layers, and multiplying many small numbers together drives it to nothing.

Residual connections

ResNet's fix, and it is one line:

$$y = f(x) + x$$

$f(x)$ is a short stack of convolutions, normalizations and activations. At the end of it you add the input back, element by element.

Why it works. Suppose the gradient through f is tiny. The gradient through the $+x$ path is exactly 1, because the derivative of x with respect to x is 1. So instead of multiplying a chain of fractions, every block has a pass-through that preserves the gradient. This is the single most important development in the history of convolutional networks; without it the depths we use now would not train.

A second benefit. The input itself is carried forward, so a later layer can use information that an earlier layer would otherwise have destroyed.

The constraint. Addition is element-wise, so the input and output must be the same shape. Adding a 64-channel tensor to a 32-channel one raises an error. Downsampling therefore happens *between* residual blocks, not inside them.

ResNet variants go to 152 layers, which was unheard of before the paper. It is over a decade old and still a standard backbone.

Dense connections

DenseNet's variation: instead of adding the input to the output, **concatenate** them. Stack the channels.

The practical difference is that concatenation does not require matching channel counts. 64 channels and 32 channels concatenate to 96 perfectly well. A later layer then uses filters to bring the channel count back down.

DenseNet is not simply better than ResNet. It is more complicated, it is better in some circumstances, and ResNet is what you will reach for by default.

Part 5: Cheaper convolutions

Grouped convolution

In a normal convolution every filter spans every input channel. In a grouped convolution you split the channels into groups and each filter only spans its own group.

Cut the channels each filter sees in half and you cut the computation in half. Into quarters, and it goes to a quarter. PyTorch exposes this directly as the `groups` argument.

Depthwise separable convolution

The extreme case, and the one with a name. Two stages:

Depthwise. Every channel gets its own single 3×3 kernel with no depth. Each filter sees exactly one channel. For 8 channels that is 8 kernels of 9 weights, so 72 weights, and the complexity drops by a factor of 8. What you lose is any mixing across channels, because nothing looks at more than one.

Pointwise. A 1×1 convolution, which is exactly the operation that mixes across channels. For 8 channels in and 8 out that is 64 weights. This puts back the fidelity the depthwise stage gave up.

Counting the whole thing for 3×3 over 8 channels: a dense convolution is 576 weights, and depthwise plus pointwise is $72 + 64 = 136$. That is a ratio of about 0.24, which is $1/9 + 1/8$. The headline figure quoted for 3×3 is roughly one ninth.

This is MobileNet's defining idea and it matters when you are short of compute, on a phone or at the edge. You give up a little accuracy for a large fraction of the cost.

Part 6: The architecture tour

You do not need to memorize layer tables. You do need to know what each one contributed, and you should be able to read an architecture table well enough to see what is happening: `conv3-64` means a 3×3 kernel with 64 filters, and the max-pool rows between blocks are where the spatial size halves.

Architecture	What it contributed
LeNet	The pattern itself: convolutions, shrinking spatial size, growing channels, then a fully connected head
AlexNet	ReLU instead of tanh, GPU training, dropout in the fully connected layers, data augmentation. Still flattens, which is wasteful

Architecture	What it contributed
VGG	Small 3×3 kernels with padding, stacked. 138 million parameters, 102 million of them in the first fully connected layer
Inception	Several kernel sizes in parallel in one block, so one layer sees several receptive field sizes at once
ResNet	Residual connections, which made real depth trainable. Still a default backbone
DenseNet	Concatenation instead of addition
MobileNet	Depthwise separable convolutions, for when compute is the binding constraint

Why several kernel sizes might help, in Inception's terms: you need the whole image to tell that it is daytime, but only one corner to see a bird in a tree. Different questions want different receptive fields.

Why 3×3 became standard: once the spatial size is shrinking anyway the receptive field grows regardless, so 5×5 buys little, and a 3×3 convolution is substantially cheaper to compute.

When choosing, the axis to think along is ImageNet accuracy against parameter count and compute. You are buying performance with resources.

Part 7: Transfer learning

Transfer learning is reusing the weights of a model trained on one task as the starting point for a different task.

The usual version: somebody trained a large backbone on ImageNet, at considerable expense. Its head does ImageNet classification, which you do not care about. Its backbone does feature extraction, which you care about a great deal. You discard the head, attach your own, and you have most of a good model for free.

If you are asked in an interview how to approach a vision task on a modest dataset, "I would build a CNN and train it from scratch" is the wrong answer. The right answer starts with a pretrained backbone.

The two ways to use it

Feature extraction. The backbone is frozen. No gradients, no updates, the weights never move. You train only the new head. Setting `requires_grad = False` is what freezes a parameter: autograd stops accumulating a gradient for it, so there is nothing for the optimizer to apply. A ResNet-18 with everything frozen and `fc` replaced by a 10-class layer trains exactly $512 \times 10 + 10 = 5{,}130$ parameters out of 11.2 million. That is Lab 6 Part 1b.

Fine-tuning. Some of the backbone is trainable, usually the last few blocks rather than all of it. **Use a very small learning rate.** The reason is worth understanding rather than memorizing: early in training your new head is random and sends back large, meaningless gradients, and a big learning rate lets those gradients knock a carefully trained backbone out of a good region of the loss surface. The pretrained information is not destroyed in some permanent sense, but you waste the advantage you started with.

Two traps in the code

Order matters when freezing. Freeze first, then replace the head. The new layer is created after the freeze, so it keeps `requires_grad = True`. Do it the other way round and the head gets frozen along with everything else, and `backward()` raises.

The optimizer has to receive the parameters you intend to train. If you unfreeze the backbone but hand the optimizer only `model.fc.parameters()`, the backbone will compute gradients and never move. Nothing raises; you have quietly done feature extraction while believing you fine-tuned.

The input contract

Each pretrained model expects its input preprocessed the way it was trained. For ImageNet models that is resize, centre crop to 224, scale to $[0, 1]$, and then normalize with `mean = [0.485, 0.456, 0.406]` and `std = [0.229, 0.224, 0.225]`.

Get the constants wrong and nothing raises. Correct normalization puts the data in roughly $[-2.12, 2.64]$; normalizing by 0.5 and 0.5 puts it in $[-1, 1]$, and every activation in the backbone is then computed on inputs it has never seen. Accuracy suffers and nothing tells you why.

The fix is not to remember the numbers. `weights.transforms()` carries the correct preprocessing with it, which is why the `Weights` enum exists.

When transfer learning does not help

A large domain gap. The early layers transfer almost always, because edges are universal. The later layers encode objects from the training set. If your inputs are not photographs of the natural world, less of that applies. Spectrograms, medical scans and remote sensing are all image-shaped, with two spatial dimensions and some channels, and we process them with the same tools, but an ImageNet backbone knows about dogs and cars, not about what you are looking for.

The input is not image-like at all. Reshaping a feature vector into a square and convolving it will run without error and will not mean anything.

You have enough labelled data of your own. With millions of labelled examples in your domain, training from scratch catches up with and then passes a frozen backbone. How much is enough has no clean answer; it depends on task complexity and how noisy the data is. Telling cats from dogs needs far less than segmenting cancer cells or extracting fields from documents.

Part 8: Backbone, neck, head

Three stages, and the middle one is optional.

Backbone. The convolutional part. Takes something like $3 \times 224 \times 224$ and returns something like $512 \times 7 \times 7$ (VGG) or $2048 \times 7 \times 7$ (ResNet-50). Pure feature extraction.

Neck. Optional. Combines or rescales feature maps from several depths of the backbone before they reach the head, so the head can see both high-resolution shallow maps and semantically strong deep ones. A feature pyramid network is the standard example, and it is covered next lecture.

Head. The task-specific part. Classification, regression, box coordinates, masks, whatever you need.

Knowing these three terms is what makes a detection network and a segmentation network look like variations on one idea rather than two unrelated architectures.

Part 9: The four tasks

Task	The question it answers	What the label looks like
Classification	What is in this image?	One class for the whole image
Detection	What is here, and where?	Per object: a class plus four numbers for the box
Semantic segmentation	Which pixels belong to which class?	An image-shaped mask, one channel per class
Instance segmentation	Which pixels belong to which *object*?	Per object: a class, a box, and a mask inside that box

Classification gives one label, which is why it cannot describe a photograph containing a cat, a dog and a second cat.

Detection needs four numbers per box. Either the two corners, $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$, or a centre with a width and height. Either way, four.

Semantic segmentation produces an output the same shape as the input. Binary segmentation needs one mask. Multi-class needs a channel per class, so height $\times$ width $\times$ classes.

Instance segmentation separates cat 1 from cat 2. To a semantic model those pixels are identical, all of them simply "cat." The practical construction is detection first, then a small mask predicted inside each detected box.

All four are the same skeleton with a different head. Detection's head does regression as well as classification, because a box centre is a continuous number. Segmentation is the one exception to "the head is fully connected": its output is an image, so it is convolutions all the way through.

Why detection is harder than classification

A network has a fixed output shape. A classifier with 10 classes always emits 10 numbers. But an image can contain three objects or thirty, and there is no way for a fixed-shape output to say "there were seventeen." Working around that mismatch is what the whole detection literature is about.

Part 10: Detection, as far as we got

Intersection over Union

The metric for how well two boxes agree. You need it both to score a prediction against a human-drawn ground-truth box and to decide when two predictions are about the same object.

$$\text{IoU} = \frac{\text{area of intersection}}{\text{area of union}}$$

Work it out in three steps. The intersection is a box too, so find its corners by taking the larger of the two left edges, the larger of the two top edges, the smaller of the two right edges and the smaller of the two bottom edges, then multiply its width by its height. If either side comes out negative the boxes do not overlap and the intersection is zero. The union is area A + area B − intersection; subtracting matters, because otherwise the overlap is counted twice.

Worked, for $A = (0, 0, 20, 20)$ and $B = (10, 10, 30, 30)$: they overlap on $x \in [10, 20]$ and $y \in [10, 20]$, so the intersection is 100. Each box is 400, so the union is $400 + 400 - 100 = 700$, and the IoU is $100/700 \approx 0.143$.

IoU is 0 for boxes that do not touch and 1 for boxes that coincide. A threshold of 0.5 is the usual convention for "these are the same object."

The region proposal idea

Two-stage detectors all do four things in order:

1. **Propose** a set of regions that might contain an object.
2. **Classify** each region independently with an ordinary CNN classifier, which also tells you what kind of object it is.
3. **Refine** the box, because the proposals are not accurate.
4. **Suppress** duplicates, because the proposal step produces many overlapping boxes for the same object.

R-CNN

The first one that matters, and now obsolete, but everything after it is built on the same four steps.

Selective search produces the proposals. It is **not a network and not trainable**: it is a classical algorithm that groups neighbouring pixels that look alike, by colour and texture, and emits a few thousand candidate regions. Replacing it with something learned is what the later models in the family are for.

Warp. The proposals come out at all shapes and sizes, and a CNN needs a fixed input shape, so each one is reshaped to the same size.

Extract features. Each warped region goes through the backbone separately. This is also R-CNN's fatal cost: a few thousand regions means a few thousand forward passes per image.

Score and refine. The features go into two outputs. One gives a confidence score and a class. The other regresses a box transformation, nudging the proposal's centre, width and height to fit the object better. Think of it as two branches off the same features. The original used an SVM for the scoring part; it was quickly replaced by a fully connected layer and nothing important changed.

Non-maximum suppression

After all that you have many overlapping boxes with scores, and you want one box per object.

The rule: **where two boxes overlap heavily, keep the one with the higher confidence.** "Heavily" means IoU above a threshold, usually 0.5. So NMS uses both quantities, confidence to decide which box wins and IoU to decide whether they are competing at all.

It is done per class. Only dog boxes can suppress other dog boxes. The reason is easy to see with an example: a mother holding a child produces a person box and a second person box with high IoU, and if you suppress across classes then a cat box can delete a dog box that is genuinely a different object. In `torchvision`, `batched_nms` takes a label per box and only suppresses within a label; plain `nms` ignores labels. In Lab 6 this is why a duplicate box over the dog survived: it was labelled `cat`, so per-class NMS left it alone.

What you should be able to do without thinking

- Say what changes between `train()` and `eval()` for BatchNorm, and why `no_grad()` is not a substitute
- Give the shape of one filter, and the number of output channels, for any convolution layer described in words
- Say how many 1×1 filters you need to go from C channels to D, and what shape each one is
- Explain why pooling and a strided convolution are not interchangeable
- Write $y = f(x) + x$ and say what the gradient of the second term is
- Say what breaks if you add tensors with different channel counts, and what DenseNet does instead
- Compute an IoU from two boxes, remembering to subtract the intersection from the union
- Name the four steps of a two-stage detector
- Say why NMS is per class

- Say which parts of a pretrained model you freeze for feature extraction, in what order the code has to go, and why the learning rate drops when you unfreeze

---

Supplemental content

Optional, and useful if a section above did not land. Prof. Hickman's centralized lecture content has pages on convolutional neural networks and transfer learning.

The lecture recording for Oct 8 is posted. If the architecture tour went past too quickly, that section runs from about 00:48:00 to 01:10:00, and transfer learning starts at about 01:13:54.